De Gouden Tarwe SQL Database Challenge (Day 4)

MySQL

Problem Statement

De Gouden Tarwe (Dutch for The Golden Wheat) is a premium bakery chain based in the Netherlands. Established in 2018, the company has grown rapidly and now operates more than 30 branches across 20 major Dutch cities including Amsterdam, Rotterdam, Utrecht, The Hague, Eindhoven, and Groningen.

The bakery sells authentic Dutch baked goods from everyday bread like tijgerbrood and volkorenbrood, to traditional pastries like stroopwafels, appeltaart, bossche bollen, and seasonal specialties like oliebollen (only sold in December) and pepernoten (only sold during the Sinterklaas season in November).

The company employs over 450 people across all branches: bakers, assistant bakers, sales staff, branch managers, delivery drivers, and support staff. It serves tens of thousands of customers, many of whom participate in a loyalty program with tiered benefits (Bronze, Silver, Gold, Platinum).

The Current Problem

De Gouden Tarwe currently stores all its operational data in flat CSV files. As the business has grown, this approach has become unmanageable:

1. Data redundancy: The same product name, price, and description are repeated in every transaction record.
2. No formal relationships: There is no way to enforce that a branch referenced in an order actually exists.
3. No data integrity: A typo in a city name creates a new city that does not actually exist.
4. Impossible to answer business questions: Simple queries like What is our best-selling product in Rotterdam? require manually cross-referencing multiple files.
5. Performance problems: With millions of rows, opening and searching CSV files is painfully slow.

Progress – Day 4

On Day 4, we created an index to serve as a table of contents for a database. The purpose of creating an index is to reduce data retrieval time, since the database doesn't need to read the entire table. Indexes are created to speed up data retrieval (SELECT), filtering (WHERE), JOIN, ORDER BY, and GROUP BY operations.

The concept of an index is similar to creating a table of content in a book. Data searching becomes faster because the system checks the table of contents first before looking into the database (like finding a page in a book). In one table, we can create more than one index. However, we don't need to create indexes for all columns. Usually, indexes are only created for columns that are frequently used in WHERE conditions. When new data is inserted, we don't need to run the index creation query again because the index will automatically update when new data comes in. However, if a very large amount of data is inserted (for example, billions of rows), sometimes the existing index needs to be dropped and rebuilt to make the

process more efficient. Here is an example of an index implementation along with its meaning from the de_gouden_tarwe database.

Index from the orders table

1. Index on the orders table, sorted by branch_id first, and within the same branch, sorted again by order_datetime

```
CREATE INDEX idx_orders_branch_date    ON orders(branch_id,
order_datetime);
```

This index works for sorting the data by branch_id, and also combination of branch_id and order_datetime. This index is a type of **composite index**. You can use this index if you want to see the orders data from a specific branch, or from a specific branch within a certain time period.

2. Index on the orders table sorted by customer_id

```
CREATE INDEX idx_orders_customer       ON orders(customer_id);
```

This index works for sorting the data by customer_id. You can use this index if you want to see the orders data from a specific customer.

3. Index on the orders table sorted by employee_id

```
CREATE INDEX idx_orders_employee       ON orders(employee_id);
```

This index works for sorting the data by employee_id. You can use this index if you want to see the orders data from a specific employee.

Index from the order_items table

1. Index on the order_items, sorted by order_id

```
CREATE INDEX idx_orderitems_order      ON order_items(order_id);
```

This index works for sorting the data by order_id. You can use this index if you want to see the order_items data from a specific order.

2. Index on the order_items, sorted by product_id

```
CREATE INDEX idx_orderitems_product    ON order_items(product_id);
```

This index works for sorting the data by product_id. You can use this index if you want to see the order_items data from a specific product.

Index from the daily_production table

1. Index on the order_items, sorted by production_date first, and within the same branch, sorted again by branch_id

```
CREATE INDEX idx_dailyprod_date        ON
daily_production(production_date, branch_id);
```

This index works for sorting the data by production_date, and also combination of production_date and branch_id. This index is a type of **composite index**. You can use this index if you want to see the daily_production data from a specific production date, or from a specific production date within a branch.

Index from the loyalty_transactions table

1. Index on the loyalty_transactions, sorted by customer_id first, and within the same branch, sorted again by transaction_date

```
CREATE INDEX idx_loyalty_customer_date ON
loyalty_transactions(customer_id, transaction_date);
```

This index works for sorting the data by customer_id, and also combination of customer_id and transaction_date. This index is a type of **composite index**. You can

use this index if you want to see the loyalty_transaction data from a specific customer, or from a specific customer within a certain time period.

Index from the waste_log table

1. Index on the waste_log, sorted by branch_id first, and within the same branch, sorted again by waste_date

```
CREATE INDEX idx_wastelog_branch_date  ON waste_log(branch_id,
waste_date);
```

This index works for sorting the data by branch_id, and also combination of branch_id and waste_date. This index is a type of **composite index**. You can use this index if you want to see the waste_log data from a specific branch, or from a specific branch within a certain time period.

2. Index on the waste_log table, sorted by product_id

```
CREATE INDEX idx_wastelog_product       ON waste_log(product_id);
```

This index works for sorting the data by product_id. You can use this index if you want to see the waste_log data from a specific product.

Index from the customer_feedback table

1. Index on the customer_feedback, sorted by branch_id

```
CREATE INDEX idx_feedback_branch        ON
customer_feedback(branch_id);
```

This index works for sorting the data by branch_id. You can use this index if you want to see the customer_feedback data from a specific branch.

Besides the indexes I mentioned above, there is another index designed to automate the deleting of old data in order to optimize storage usage. It's called Index Lifecycle Management (ILM). We can set up an index to store data for a specific period of time, and it will be automatically deleted once that period has passed. ILM is actually a feature of Elasticsearch, but we can implement ILM manually in MySQL using partitioning concept.